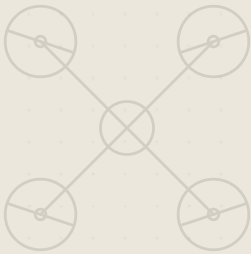


Six EE Projects That Think For Themselves

Rovers, arms, legs, eyes, and hands — six complete robots that run the loop every real robot runs: sense, decide, act. From a rover that avoids walls on its own, to an arm that solves its own joint angles, to a quadruped that walks, to a rover that maps a room it has never seen. Commodity parts; real robotics engineering.

01	Reactive Rover A rover that senses walls and steers itself — the sense-plan-act loop in its simplest form	\$35
02	Robot Arm with Inverse Kinematics Tell the arm an X, Y, Z and it works out every joint angle itself — pick and place, from math	\$60
03	Quadruped Walker Four legs, twelve servos, and a gait that actually walks — the robot everyone stops to watch	\$90
04	Vision-Guided Tracking Robot A robot that sees a target and follows it across the room — perception meets control	\$45
05	Force-Sensing Gripper A gripper that feels how hard it's squeezing — grab an egg without breaking it	\$40
06	SLAM Mapping Rover A rover that maps a room it has never seen, then navigates it — autonomy, for real	\$95

● By Enes · @1nska · Electrical Engineer — a Build List field guide



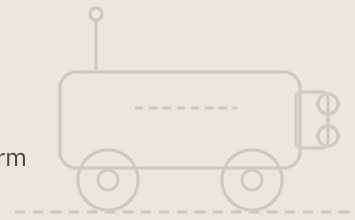
Reactive Rover

A rover that senses walls and steers itself — the sense-plan-act loop in its simplest form

\$35

beginner

1 weekend



// why this matters

Every robot, from a vacuum to a Mars rover, runs the same loop: sense the world, decide what to do, act, repeat. A reactive rover is the purest version — an ultrasonic and a pair of IR sensors feed a few simple rules that steer two motors, and the machine roams a room avoiding walls and furniture with no map and no plan. It is the cheapest way to build something that behaves on its own, the payoff is immediate (it just drives off), and it is the chassis and control skeleton every later project in this guide builds on.

WHAT THIS PROVES TO A RECRUITER

You understand the sense-plan-act loop, distance sensing (ultrasonic time-of-flight, IR reflectance), differential-drive steering, and behavior-based control. That is the foundation of mobile robotics and embedded control.

→ **the job this maps to:** mobile robotics & embedded systems — robot-vacuum, warehouse-AMR, and consumer-robotics teams.

// how it works

- ▶ **Sense-plan-act:** The robot reads its sensors, applies a rule to decide a motion, drives the motors, and repeats many times a second. The entire behavior emerges from that tight loop — no global map required.
- ▶ **Ultrasonic ranging:** An HC-SR04 emits a 40 kHz ping and times the echo; distance is the speed of sound times the time, halved. It gives you a “how far to the thing ahead” number tens of times a second.
- ▶ **IR reflectance:** Cheap IR sensors detect a nearby obstacle (or a line or edge) by bounced infrared. Mounted at the corners, they catch what the forward ultrasonic misses.
- ▶ **Differential drive:** Two independently driven wheels (plus a caster) steer by speed difference: equal speeds go straight, unequal speeds turn, opposite spin turns in place. It is the simplest steerable platform there is.
- ▶ **Behavior rules:** A priority list of simple behaviors — if something is close ahead, back up and turn; else if a side sensor fires, veer away; else drive forward — produces surprisingly lifelike wandering. This is behavior-based (subsumption) control.

// bill of materials

COMPONENT	PART / SPEC	QTY	~COST
Chassis + motors + caster	2WD robot chassis kit	1	\$12
Motor driver	TB6612 / L298N	1	\$4
Ultrasonic sensor	HC-SR04	1	\$2
IR obstacle sensors	reflectance modules (×2)	2	\$3
Microcontroller + battery	Arduino / ESP32, 18650 or AA pack	1	\$14

// build steps

1 Build and drive the chassis

Assemble the 2WD chassis, wire the motor driver, and drive forward, back, and turn from code. Confirm the wheels spin the right way.

2 Read the ultrasonic

Mount the HC-SR04 facing forward, trigger a ping, time the echo, and print distance. Wave a hand in front and watch the number track.

3 Add the IR sensors

Mount IR sensors at the front corners and read them. Confirm they trigger on a nearby wall or hand.

4 Write the behavior rules

Implement the priority rules: close ahead means back up and turn, a side sensor means veer, otherwise forward. Let it loose in a room and watch it avoid furniture.

5 Tune and make it lifelike

Adjust thresholds, turn durations, and speeds until it roams smoothly without getting stuck. Add a random turn direction so it explores — your first autonomous robot.

// interview questions

- What is the sense-plan-act loop, and why is it fundamental to robotics?
- How does an ultrasonic sensor measure distance?
- How does a differential-drive robot steer?
- What is behavior-based (subsumption) control?
- Why can a reactive robot navigate without a map?
- What are the failure modes of ultrasonic and IR sensing?

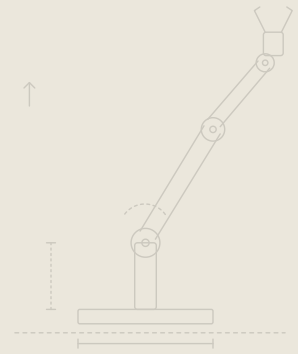
Robot Arm with Inverse Kinematics

Tell the arm an X, Y, Z and it works out every joint angle itself — pick and place, from math

\$60

intermediate

1-2 weekends



// why this matters

A robot arm looks simple until you try to move its hand to a specific point in space: you have to solve, in reverse, what every joint angle must be to put the gripper there. That is inverse kinematics, and it is the difference between a servo puppet and a robot you command in coordinates. Build a 4-axis arm, derive its geometry, implement IK, and you can type a position and watch it reach there, then pick up an object and place it somewhere else. It is the classic centrepiece of robotics, the flagship of this guide, and a direct lesson in the math every industrial arm runs.

WHAT THIS PROVES TO A RECRUITER

You understand forward and inverse kinematics, joint-space versus Cartesian-space control, servo coordination, workspace limits, and trajectory smoothing. Kinematics and manipulation are core to industrial and collaborative robotics.

→ **the job this maps to:** robotics and manipulation engineering — industrial-robot, cobot, and automation teams.

// how it works

- ▶ **Forward kinematics:** Given the joint angles and link lengths, trigonometry tells you where the gripper ends up. It is the easy direction, and it is how you verify your model against the real arm.
- ▶ **Inverse kinematics:** Given a target (x, y, z), you solve backward for the joint angles. For a 4-axis arm: base rotation from the target's bearing, then a planar two-link triangle (shoulder and elbow) solved with the law of cosines, then the wrist to level the gripper.
- ▶ **The two-link triangle:** The shoulder, elbow, and target form a triangle whose sides are your link lengths and the distance to the target. The law of cosines gives the elbow angle; a little more trig gives the shoulder. That is the heart of the solution.
- ▶ **Joint limits and workspace:** Servos only rotate so far and links have fixed lengths, so only a region of space is reachable. Your IK must check reachability and respect limits, or you command the impossible.
- ▶ **Smooth motion:** Jumping servos straight to a new angle is jerky. Interpolating between poses over time (in joint space, or along a straight Cartesian path) makes motion smooth and predictable — the beginning of trajectory planning.

// key design decisions

- **Why solve inverse kinematics instead of driving joints by hand?** Joint angles are not how tasks are described — a task says “put the gripper here.” IK translates a Cartesian target into joint angles automatically, so you can command positions, follow paths, and place objects precisely. Without it, an arm is a puppet; with it, it is programmable.
- **Why a closed-form geometric solution instead of a numeric solver?** A 4-axis arm with a planar geometry has a clean trigonometric solution: rotate the base to the target’s bearing, then solve the two-link triangle with the law of cosines. It is exact, instant, and easy to reason about — and it teaches the geometry. Numeric solvers matter for six or more axes and redundancy.
- **Why must IK check reachability?** The arm can only reach points within its link lengths and joint limits; some targets are too far, too close, or need an impossible elbow angle. IK must validate the solution and clamp or reject bad targets, or the arm slams into itself — the practical discipline real controllers enforce.

// bill of materials

COMPONENT	PART / SPEC	QTY	~COST
4-axis arm frame	servo arm kit (3D-printed or acrylic)	1	\$25
Servos	MG996R / DS3218 (×4) + gripper servo	5	\$22
Servo driver	PCA9685 16-channel PWM board	1	\$5
Microcontroller + power	ESP32, 5–6 V servo supply	1	\$8

// build steps

1 Assemble and centre the arm

Build the arm, mount servos at their mid-positions, and drive each joint individually. Confirm range and direction for base, shoulder, elbow, and wrist.

2 Measure and model

Measure your link lengths and joint offsets, and write forward kinematics. Command known angles and check the gripper lands where the math predicts.

3 Implement inverse kinematics

Solve the base bearing, the two-link triangle with the law of cosines, and the wrist. Command an (x, y, z) and watch the arm reach it. Fix the sign errors — there will be some.

4 Add limits and smoothing

Reject unreachable targets, clamp joint limits, and interpolate between poses so moves are smooth. Trace a straight line and a circle in the air.

5 Pick and place

Add the gripper, command a sequence — move above an object, descend, grip, lift, move, release — and place objects from one spot to another. Film it from the side.

// interview questions

- What is the difference between forward and inverse kinematics?
- How do you solve inverse kinematics for a two-link planar arm?
- Why is the law of cosines useful in arm kinematics?
- What is a robot's workspace, and why must IK check reachability?
- Why is interpolation needed for smooth arm motion?
- When would you need a numeric IK solver instead of a closed-form one?

Quadruped Walker

Four legs, twelve servos, and a gait that actually walks — the robot everyone stops to watch

\$90

advanced

2 weekends



// why this matters

Nothing stops a scroll like a small four-legged robot walking across a table. Each leg is a little robot arm with its own inverse kinematics, and walking is the art of moving them in a coordinated rhythm — a gait — while keeping the body balanced over the feet still on the ground. Build a 12-servo quadruped, solve leg IK, and implement a creep or trot gait, and you have a machine that walks, turns, and stands up on its own. It is the most ambitious build in this guide, a genuine mechatronics and control challenge, and the same fundamentals behind every legged robot from research labs to robot dogs.

WHAT THIS PROVES TO A RECRUITER

You understand leg inverse kinematics, gait generation and phasing, static stability (support polygons), servo torque budgeting, and coordinating many actuators in real time. Legged locomotion is one of the most demanding — and most exciting — corners of robotics.

→ **the job this maps to:** legged robotics and mechatronics — robot-dog, humanoid, and research-lab robotics teams.

// how it works

- ▶ **Legs are arms:** Each leg has three joints — hip abduction, hip pitch, knee — and placing a foot at an (x, y, z) is the same inverse-kinematics problem as the arm, solved twelve times over for four legs.
- ▶ **Gaits:** A gait is the pattern of which legs are in the air and which are on the ground. A creep moves one leg at a time (slow, always stable); a trot moves diagonal pairs together (faster, needs balance). Phasing the legs correctly is what makes walking work.
- ▶ **Stability:** When a leg lifts, the body must stay balanced over the remaining feet. The support polygon — the area under the grounded feet — must contain the centre of mass, or the robot tips. Shifting the body before lifting a leg is the trick.
- ▶ **Swing and stance:** A stepping leg swings forward through the air; the grounded legs stroke backward together to push the body forward. Coordinating swing and stance across all legs, cycle after cycle, is the gait engine.
- ▶ **Torque and timing:** Twelve servos moving under load draw serious current and must move smoothly and in sync. A dedicated supply, sensible speeds, and a fixed-rate control loop are the difference between a walk and a twitching heap.

BUILD NOTE

Print a proven open-source quadruped frame rather than designing your own, use decent metal-gear servos (cheap plastic ones strip under load), and power the servos from a dedicated high-current supply. Get one leg's IK working standing still before you attempt a single step.

// bill of materials

COMPONENT	PART / SPEC	QTY	~COST
Quadruped frame	3D-printed open-source design	1	\$15
Servos	metal-gear 20 kg class (×12)	12	\$45
Servo driver	PCA9685 (16-channel)	1	\$5
Microcontroller	ESP32 (control loop, remote)	1	\$8
Power	2S LiPo + high-current 6 V regulator	1	\$17

// build steps

1 Build and calibrate one leg

Assemble a single leg, mount its three servos, and calibrate their zero positions. Drive each joint and confirm range.

2 Solve leg IK

Implement inverse kinematics for one leg and command foot positions. Move the foot through a square in the air and confirm it tracks. Then replicate to all four legs.

3 Stand up

Command all four feet to a standing stance and lift the body off the ground. Balance the stance and tune the servo speeds so it rises smoothly.

4 Implement a creep gait

Move one leg at a time — shift the body over the other three, swing one leg forward, set it down, repeat. Get it to take slow, stable steps forward.

5 Trot and turn

Move to a diagonal-pair trot for speed, add turning by asymmetric strokes, and add a remote for direction. Now walk it across the table and film it.

// interview questions

- How is a robot leg an inverse-kinematics problem?
- What is a gait, and how do a creep and a trot differ?
- What is the support polygon, and why does it determine stability?
- Why must the body shift before a leg is lifted in a creep gait?
- Why do legged robots demand high-torque servos and a dedicated supply?
- How do you coordinate many servos in a real-time control loop?

Vision-Guided Tracking Robot

A robot that sees a target and follows it across the room — perception meets control

\$45

intermediate

1-2 weekends



// why this matters

A robot that can see is a robot that can do almost anything, and the simplest vision task — find a thing and follow it — teaches the whole perception-to-action pipeline. Put a camera on your rover or a pan-tilt head, detect a coloured object (or a face) with OpenCV, and steer the robot to keep it centred and at a set distance. The result follows you around a room, and the pipeline (capture, detect, compute error, control) is exactly what robot vacuums, delivery robots, and industrial vision systems run. It is the bridge from blind reactive behavior to robots that understand what is in front of them.

WHAT THIS PROVES TO A RECRUITER

You understand image capture and processing, object detection (colour segmentation, face and feature detection), turning detection into a control error, visual servoing, and the latency and robustness issues of real-world vision. Perception plus control is the core of modern robotics.

→ **the job this maps to:** robot perception and computer-vision engineering — autonomous-vehicle, delivery-robot, and industrial-vision teams.

// how it works

- ▶ **Capture and detect:** A camera grabs frames; OpenCV finds the target — simplest via colour segmentation (threshold in HSV for a coloured ball), or a face or feature detector. The output is a bounding box or centroid in pixel coordinates.
- ▶ **From pixels to error:** The target's horizontal offset from image centre is your steering error; its apparent size is your range error. Perception has become two numbers a controller can use.
- ▶ **Visual servoing:** Feed those errors into control loops: turn toward the target to zero the horizontal error, drive forward or back to hold the size at a setpoint. The robot now tracks and follows — a control loop closed through a camera.
- ▶ **Latency and robustness:** Vision is slow and noisy compared to a sensor: frames take time, detection jitters, lighting changes ruin colour thresholds. Filtering the detection and tuning gains for the latency are what make tracking smooth.
- ▶ **Stepping up:** Swap colour segmentation for a lightweight neural detector (a MobileNet-class model on a Pi) and the same pipeline tracks people or objects by class — the path to real-world perception.

// bill of materials

COMPONENT	PART / SPEC	QTY	~COST
Camera + compute	Raspberry Pi + camera, or ESP32-CAM	1	\$30
Mobile base or pan-tilt	project-1 rover, or pan-tilt servos	1	\$10
Motor driver / servos	as needed	1	\$4
Power	battery pack	1	\$1

// build steps

1 Stream and view video

Set up the camera and confirm you can capture frames and display them (or stream to a laptop). Check the frame rate — it sets your control rate.

2 Detect a coloured target

Threshold in HSV to segment a brightly coloured ball, find its contour, and draw its centroid and bounding box. Tune the thresholds for your lighting.

3 Compute errors

Calculate horizontal offset from centre and target size each frame. Print both and move the ball around to see them respond.

4 Close the loop

Drive the base (or pan-tilt) to zero the offset and hold the size at a setpoint. Tune gains until it tracks smoothly without oscillating.

5 Follow you

Let it follow the ball (or you) around the room. Test changing light and speed, add filtering, then try a face or neural detector for a bigger challenge.

// interview questions

- How does colour segmentation detect an object in an image?
- How do you turn a detected object's position into a control error?
- What is visual servoing?
- Why is latency a bigger problem for vision-based control than sensor-based control?
- What makes colour-based detection fragile, and how do you improve it?
- How would you extend simple tracking to detecting people or objects by class?

Force-Sensing Gripper

A gripper that feels how hard it's squeezing — grab an egg without breaking it

\$40

intermediate

1 weekend



// why this matters

A gripper that only knows its finger position will crush a strawberry and drop a coffee cup. Give it a sense of touch — a force sensor in the fingertip and a control loop on grip force — and it can pick up an egg, a paper cup, or a tool with just the right squeeze. Building one teaches force sensing, compliant mechanisms, and force control as opposed to position control, which is the difference between industrial pick-and-place and robots that can handle the messy, fragile real world. It is a compact, satisfying build with an unforgettable demo.

WHAT THIS PROVES TO A RECRUITER

You understand tactile and force sensing (FSRs, load cells), compliant gripper design, closed-loop force control versus position control, grasp detection, and the mechanics of gentle manipulation — core to manipulation and soft-robotics engineering.

→ **the job this maps to:** manipulation and gripper engineering — warehouse-picking, food-handling, and collaborative-robot teams.

// how it works

- ▶ **Sensing touch:** A force-sensitive resistor (FSR) drops in resistance as you press it; a load cell flexes and changes a strain-gauge bridge. Either gives you a voltage that rises with grip force at the fingertip.
- ▶ **Position vs force control:** A position-controlled gripper closes to a set opening regardless of what is in it — fine for rigid parts, disastrous for fragile ones. Force control closes until the fingertip force reaches a setpoint, then holds, adapting to any object.
- ▶ **Compliance helps:** Flexible fingers (fin-ray or soft pads) conform to an object's shape, spreading contact and forgiving errors. Mechanical compliance does half the work before the control loop even acts.
- ▶ **Grasp detection:** A rising force signal tells you contact happened; a stable force tells you the grasp is secure; a sudden drop tells you the object slipped. Reading these events is how a robot knows what its hand is doing.
- ▶ **The control loop:** Command the servo to close, read fingertip force, and stop (or servo around) at the target force. Set a low target for an egg, a higher one for a tool — the same gripper, different objects.

BUILD NOTE

A servo-driven parallel or fin-ray (compliant) gripper printed from an open design is enough. Put the force sensor in a fingertip pad, and remember the sensor reads a squeeze, not a mass — you are controlling grip force, not measuring weight.

// bill of materials

COMPONENT	PART / SPEC	QTY	~COST
Gripper mechanism	3D-printed parallel / fin-ray gripper	1	\$10
Servo	MG996R or similar	1	\$6
Force sensor	FSR 402, or small load cell + HX711	1	\$8
Microcontroller	Arduino / ESP32	1	\$8
Pads + wiring	soft fingertip pads, passives	—	\$8

// build steps

1 Build the gripper

Print and assemble a parallel or fin-ray gripper on a servo. Drive it open and closed and confirm smooth travel.

2 Add the force sensor

Mount the FSR (or load cell) in a fingertip pad and read it. Press with your finger and watch the value rise with pressure.

3 Calibrate force

Press with known weights (or a kitchen scale) to map sensor reading to grip force. Note its nonlinearity and set a usable range.

4 Close the force loop

Close the gripper until the fingertip force hits a setpoint, then hold. Test on a rigid block, then a paper cup, adjusting the setpoint.

5 The egg test

Set a gentle target and pick up an egg, then a strawberry, then a tool with a higher target. Add slip detection (a force drop) to re-grip. Film the egg.

// interview questions

- How does a force-sensitive resistor sense grip force?
- What is the difference between position control and force control of a gripper?
- Why does mechanical compliance help with grasping?
- How can a gripper detect contact, a secure grasp, and slip?
- Why is a fingertip force sensor measuring squeeze rather than weight?
- How would you scale this to multi-finger or soft grippers?

SLAM Mapping Rover

A rover that maps a room it has never seen, then navigates it — autonomy, for real

\$95

advanced

2–3 weekends



// why this matters

The rover in project 1 wanders blind; this one knows where it is. Put a spinning LiDAR on a wheeled base, add wheel odometry, and run SLAM — simultaneous localisation and mapping — and the robot builds a map of the room as it drives while tracking its own position inside it. Then give it a goal and it plans a path around the furniture and drives there. This is the technology in every robot vacuum and warehouse robot, and building it end to end is the capstone of this guide: perception, estimation, mapping, and planning in one machine.

WHAT THIS PROVES TO A RECRUITER

You understand odometry and its drift, LiDAR scanning, SLAM at a conceptual and practical level (scan matching, pose estimation, occupancy grids), and path planning on a real robot — the complete autonomy stack of mobile robotics, and one of the most in-demand skill sets in the field.

→ **the job this maps to:** autonomous-navigation and SLAM engineering — robot-vacuum, warehouse-AMR, delivery-robot, and autonomy teams.

// how it works

- ▶ **Odometry, and why it drifts:** Counting wheel rotations gives you a position estimate, but wheel slip and small errors accumulate, so odometry alone wanders off within metres. It is a good short-term guess and a bad long-term truth.
- ▶ **LiDAR scans the room:** A spinning 2D LiDAR returns hundreds of range measurements per revolution — a slice of the surrounding walls. Each scan is a snapshot of what the world looks like from where the robot is.
- ▶ **SLAM: scan matching:** SLAM compares each new scan to the map so far, finds the pose that best aligns them, corrects the odometry drift, and adds the scan to the map. Localisation and mapping improve each other — the “simultaneous” in SLAM.
- ▶ **The occupancy grid:** The map is a grid where each cell is free, occupied, or unknown, built up from the scans. That grid is what the planner reasons over.
- ▶ **Planning and navigation:** Given the map and a goal, a planner (A* over a costmap that inflates obstacles) finds a path, and a local controller drives it while dodging anything new. The robot is now autonomous in a space it built its own model of.

BUILD NOTE

Use the tooling: a Raspberry Pi running ROS 2 with a cheap 2D LiDAR (RPLidar A1 or LD19) and the `slam_toolbox` and `Nav2` packages gets you a working map and navigation without writing SLAM from scratch. Learn what the packages do first; go deeper on the algorithms once it drives.

// bill of materials

COMPONENT	PART / SPEC	QTY	~COST
2D LiDAR	RPLidar A1 or LD19	1	\$50
Compute	Raspberry Pi 4 / 5 (ROS 2)	1	\$25
Base with encoders	2WD chassis + encoder motors + driver	1	\$15
Power	battery + regulators	1	\$5

// build steps

1 Build the base with odometry

Assemble a 2WD base with encoder motors, publish wheel odometry, and drive it in a square. Watch how the odometry drifts from where the robot actually ends up.

2 Mount and read the LiDAR

Mount the LiDAR level on top and stream scans. Visualise a scan and confirm the room's walls appear as you turn the robot.

3 Run SLAM

Launch `slam_toolbox` with odometry and LiDAR, drive the robot slowly around the room, and watch the map build in real time. Note how the pose stays locked even as odometry drifts.

4 Save the map and localise

Save the finished map, restart, and have the robot localise itself within it from its scans alone. This is the moment it “knows where it is.”

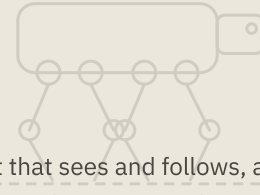
5 Navigate to a goal

Launch `Nav2`, click a goal on the map, and watch the robot plan a path around obstacles and drive there. Drop something in its way and watch it re-plan — autonomy achieved.

// interview questions

- What is odometry, and why does it drift?
- What does SLAM solve, and why are mapping and localisation coupled?
- How does scan matching correct a robot's pose estimate?
- What is an occupancy grid?
- How does a planner find a path through a costmap?
- What are the main failure modes of LiDAR SLAM in real environments?

Every robot is the same loop: sense, decide, act.



A rover that avoids walls, an arm that reaches a coordinate, a quadruped that walks, a robot that sees and follows, a hand that feels, a rover that maps its own world — six robots, one loop, escalating from reflexes to real autonomy. The parts are commodity (servos, a LiDAR, a Pi); the skill is in the integration — the kinematics, the control loops, the perception, the estimation. That is exactly what robotics teams hire for: people who can make the whole loop work on real hardware.

// the skill ladder

Build them in order and each one hands you the technique the next one needs:

reactive control → kinematics → locomotion → perception → force control → SLAM & navigation

! BEFORE YOU POWER IT UP

Servos and motors pinch and draw serious current, and LiPo batteries need respect. Set current limits, keep fingers clear of legs and grippers under power, never leave a LiPo charging unattended, power down before you rewire, and test the quadruped and rovers in a clear area away from stairs and pets.

DO THIS

Build even one. Film it — a rover roaming, an arm placing an object, a quadruped's first steps, a gripper lifting an egg, a map drawing itself — and put the code on GitHub with your kinematics, your tuning, and an honest note on what works and what doesn't. One robot demo and one clean repo prove you can make the whole loop work.

Full build notes, parts links, and the next field guide go out in **The Build List**. The people actually shipping these — and trading IK and gait tricks — are in **Ohm's Club**.

— Enes (@1nska) · Electrical Engineer